Steganography Study Summary

LSB and its weaknesses, JPEG steganography (JSteg → F3 → F4 → F5), matrix embedding / Hamming codes, and JPEG-robust methods. Based only on the four uploaded papers; anything beyond them is flagged explicitly.

References (cited in the text as [1]–[4])

- [1] N. Subramanian, O. Elharrouss, S. Al-Maadeed, A. Bouridane, "Image Steganography: A Review of the Recent Advances," IEEE Access, vol. 9, pp. 23409–23423, 2021.
- [2] J. Fridrich, T. Pevný, J. Kodovský, "Statistically Undetectable JPEG Steganography: Dead Ends, Challenges, and Opportunities," ACM MM&Sec '07, Dallas, 2007.
- [3] Z. Bao, X. Luo, Y. Zhang, C. Yang, F. Liu, "A Robust Image Steganography on Resisting JPEG Compression with No Side Information," IETE Technical Review, vol. 35, no. S1, pp. 4–13, 2018.
- [4] L. Fan, T. Gao, Q. Yang, Y. Cao, "An extended matrix encoding algorithm for steganography of high embedding efficiency," Computers and Electrical Engineering, vol. 37, pp. 973–981, 2011.

1. What steganography is, and how it differs from cryptography

Steganography is the art of hiding the **existence** of a secret message by embedding it in an innocent-looking cover object. According to [4], the classic model is Simmons' 1983 "Prisoners' Problem": two prisoners must communicate without making the guard suspicious.

	Cryptography	Steganography
What is hidden	The **content** of the message (made unreadable)	The **existence** of the message
Output	Ciphertext, visible in scrambled form, which can attract suspicion and scrutiny [1]	An innocent-looking cover/stego object
When it is "broken"	When the content is decrypted	As soon as the presence of a message is detected, even if the content stays hidden [2], [4]
Counterpart	Cryptanalysis	Steganalysis (is it a stego image? where and what is the content?) [1]

Difference from watermarking: [1] states that steganography stresses secrecy while watermarking emphasizes robustness. [3] adds that robust watermarking algorithms do not aim at resisting steganalysis, so they cannot simply be used as steganography.

Steganographic security (Cachin): If covers have distribution Pc and stego objects have distribution Ps, security is measured with the Kullback–Leibler distance $D(Pc\|Ps)$. Schemes with $D < \varepsilon$ are called ε-secure; $D = 0$ is perfectly secure [2]. Since the true distribution of natural images is unknown, [2] instead measures the minimal average error of a trained classifier (SVM): $P = (P_{FA} + P_{MD}) / 2$. $P = 50\%$ means the detector is guessing.

2. The general process: embedding and extraction

Formal definition from [2]: cover $x \in C$, secret message $m \in M$, secret key $k \in K$.

- **Embedding:** `y = Emb(x, m, k)` gives the stego object y.
- **Extraction:** `Ext(y, k) = m` recovers the message when the correct key is used.

Figure 1 of [1] shows the same flow: the sender feeds the cover and secret into an embedding algorithm; the receiver applies an extraction algorithm to the stego image; steganalysis is the detection attempt by an observer.

Typical steps (compiled from the flows in [2], [3], [4]):

1. Choose the cover (e.g. grayscale JPEG at QF 70 in [2]).
2. Convert the message to bits; optionally add error correction (e.g. Reed–Solomon in [3]).
3. Choose the **embedding domain**: pixel LSBs (spatial) or quantized DCT coefficients (transform).
4. Choose the **embedding positions**: sequential, pseudo-random path driven by the key, or a content-adaptive selection rule [2].
5. Apply the **embedding operation** (LSB flip, ±1 change, etc.), often together with a coding scheme (matrix embedding, wet paper codes, STC).
6. Transmit the stego object.
7. The receiver runs extraction with the same key/code.

Metrics

- **Capacity:** $\log_2|M|$ bits [2]. Units: bpp = $L/(H \cdot W \cdot C)$ [1]; for JPEG, bpac (bits per non-zero AC coefficient) [2].
- **Embedding efficiency:** $e = \log_2|M| / E\{d(x, y)\}$, i.e. bits embedded per unit distortion [2]. In [4]: average number of embedded bits per change.
- **Visual quality:** MSE and PSNR = $10 \cdot \log_{10}(R^2/MSE)$, with $R = 255$ for 8-bit images [1].

- **Security:** steganalysis accuracy or detection error P [1], [2].

These goals conflict: higher capacity usually costs secrecy, and methods robust to noise usually sacrifice capacity or secrecy [1].

3. Spatial domain vs transform domain

	Spatial domain	Transform domain (JPEG/DCT)
Where the message goes	LSBs of pixel values	Quantized DCT coefficients
Examples	LSB, PVD [1]	JSteg, F3/F4/F5, Steghide, MBS [2], [4]
Weakness	Vulnerable to pixel alteration and visual attack [4]	Coefficient histograms and block dependencies can reveal changes [2]
Strength	Simple, high capacity	Less conspicuous areas; JPEG is the most common format, so it gets the most attention [2], [4]

JPEG pipeline [3]: color-space conversion → down-sampling → 8×8 blocking → block DCT → quantization of the coefficients by the quality factor (QF) to integers → encoding. Adaptive JPEG steganography modifies these rounded, quantized DCT coefficients [3]. F5 embeds only into non-zero AC coefficients [2]; MBS builds models for the 63 AC modes [2].

4. LSB steganography and its weaknesses

How it works. The least significant bits of the cover pixels are replaced with the secret bits, on the assumption that modifying a few pixel values will not show any visible change [1]. The LSB plane looks random and noisy, which is why it is used [4].

The math. If message bits and cover bits are uniformly distributed, a position is changed with probability 0.5 on average, so embedding efficiency is 2 bits per change. If the whole image is filled, 50% of the carrier data changes, which makes statistical steganalysis likely to succeed [4].

Weaknesses (all from the papers):

1. **Overloading becomes visible.** Filling the cover too much may lead to visible changes [1].

2. **Spatial-domain methods are vulnerable** to pixel alteration and visual attack [4].
3. **The embedding operation itself is detectable.** [2] attributes the high detectability of LSB-flipping methods to the flipping of LSBs itself (citing Ker's structural analysis).
4. **Pairs-of-values (POV) problem.** LSB replacement pushes the counts of the two values in each pair (2k, 2k+1) toward each other. In JSteg this leaves a clear histogram artifact, detectable by Westfeld and Pfitzmann's χ^2 test [4].
5. **Detection at very low rates.** JSteg-style embedding can be reliably detected at rates as low as 0.05 bits per usable coefficient with the generalized category attack [2].
6. **Lossy channels destroy the message.** JPEG compression often destroys the message hidden by traditional steganography [3].
7. **Limited capacity and security.** Traditional methods (LSB and variants) are less secure than deep learning methods and have limited capacity; using more pixels for the message leads to distortions [1].

Worked example (illustration of the LSB rule). Pixels 150, 201, 78 (binary 10010110, 11001001, 01001110), message bits 1, 0, 1 → new pixels 151, 200, 79. Extraction reads the LSBs 1, 0, 1. Each value changes by at most 1.

LSB variants (traditional-methods section of [1]):

- **Huffman + LSB:** the message is Huffman-encoded first, then embedded with LSB.
- **k-LSB:** the k least significant bits are replaced by the message.
- **RGB LSB:** bit-slicing over three channels, with a 2:2:4 ratio for R, G, B.
- **LSB + PVD (Pixel Value Differencing):** the difference of consecutive pixels decides where and how much to hide; for every 8 bits, LSB on the first two bits and PVD on the remaining six.
- **LSB with dynamic key:** in [1] this is the work of Patel & Meena: a pseudo-random number generator selects the pixels and the key is encrypted with a rotation every time. The idea is "key-dependent random positions" instead of sequential embedding. It is the same logic as JSteg moving from a sequential to a pseudo-random path [2] and Outguess scattering bits with a PRNG [4].

Note: "dynamic LSB" is not defined as one standard algorithm in these papers; present it with the meaning above and cite [1].

5. JPEG steganography: JSteg → Outguess → F3 → F4 → F5

The history is a cat-and-mouse game: each method tries to fix the point where the previous one was caught.

JSteg

- Embeds message bits as the LSBs of quantized DCT coefficients that are different from 0 and 1. These two values form an LSB pair, and allowing changes at 0 would introduce a large distortion [2].
- The embedding path was originally sequential, later extended to a pseudo-random path [2].
- **Problem:** it introduces characteristic artifacts into the first-order statistics (histogram) of DCT coefficients [2]; the POV problem is caught by the χ^2 test [4]; the histogram attack and category attacks work even at low rates [2].

Outguess

- **v0.1:** scatters the embedding locations over the whole DCT domain with a pseudo-random generator. The χ^2 test initially failed, but steganalysts then found a flaw that leaves statistical artifacts [4].
- **v0.2:** tries to keep the statistical properties of the cover after embedding (statistical restoration) [4]. The idea: split the cover into two disjoint parts, embed in one, and use the other to make corrections that preserve the histogram [2].
- **Problem:** Fridrich et al. use the discontinuity at the border between two adjacent 8×8 blocks to estimate the length of the hidden data [4]. Restoring only the histogram is not enough because DCT coefficients have many inter- and intra-block dependencies; the extra distortion can even make the method more detectable [2].

F3

- Compares the absolute value of the coefficient with the secret bit. If they match, nothing changes; otherwise the absolute value is reduced by 1 [4].
- **Solves** the POV problem of JSteg [4].
- **Problem:** more even coefficients than odd ones are introduced, so the histogram looks obviously unnatural [4].
- Illustration of the rule: coefficient 3 with bit 1 stays 3; −2 with bit 1 becomes −1; 5 with bit 0 becomes 4; 1 with bit 0 becomes 0 (the coefficient vanishes: see shrinkage under F5).

F4

- Assigns a bit value to each coefficient by its sign: even-negative and odd-positive coefficients represent 1; odd-negative and even-positive represent 0 [4].
- If the secret bit equals the coefficient's symbol, no change; otherwise the absolute value is reduced by 1 [4].

- Result: the histogram of the stego image looks statistically similar to a clean image [4].
- Illustration: 4 (symbol 0) with bit 1 becomes 3 (symbol 1); −2 (symbol 1) with bit 0 becomes −1 (symbol 0).

F5 (Westfeld)

F5 is developed on top of JSteg, F3 and F4 [4]. Three main ingredients:

1. **Embedding operation:** the absolute value of the coefficient is always decreased by 1 instead of flipping LSBs; the author correctly attributed high detectability to the LSB flipping itself [2]. Only non-zero AC coefficients are used [2].
2. **Matrix embedding (syndrome coding)** with binary Hamming codes: more bits per change [2], [4] (see section 6).
3. **Permutative straddling:** scatters the message over the whole carrier, which enhances security [4].

Shrinkage. If a coefficient equal to 1 or −1 is decreased, it becomes 0. The receiver reads only non-zero coefficients, so it would skip it; therefore the same bit is embedded again at the next coefficient. This lowers efficiency and affects the histogram. F5 skips about 50% of the ±1 coefficients to reduce the impact [2].

F5 variants (experiments in [2]):

- **−F5:** increases instead of decreases the absolute value to avoid shrinkage. Surprisingly it is worse than F5, which validates the original design choice.
- **nsF5 (no-shrinkage F5):** removes shrinkage with wet paper codes; the best tested algorithm that does not use side information at the sender.

Note: F1/F2 are not mentioned in these papers; the narrative starts at F3 [4]. Check with your advisor what "F1/F2" refers to.

6. Matrix embedding and Hamming codes

Idea. Let x be the LSB vector of n cover coefficients, m the k-bit message, H a k×n binary matrix. Compute the syndrome `Hx`. If `Hx = m`, no change is needed. Otherwise, `Hx ⊕ m` equals exactly one column of H (the j-th); flipping bit j of x fixes it. The receiver reads the message as the syndrome `m = Hy` [2].

n = 7, k = 3 example (the H of [2]):

```
H = 1 0 0 1 1 0 1
    0 1 0 1 0 1 1
    0 0 1 0 1 1 1
```

The columns of H are all non-zero 3-bit vectors (1 to 7 in binary).

Example: cover LSBs x = 1011001, message m = 011.

- `Hx = (1,0,0)`, which is not equal to `m = (0,1,1)`.
- `Hx ⊕ m = (1,1,1)` = column 7 of H, so flip bit 7 of x.
- y = 1011000 and `Hy = (0,1,1) = m` ✓. Three bits are embedded in seven positions with a single change.

Hash-function notation (Fan et al.) [4]. The same thing is written as `f(a) = ⊕ (a_i · i)`, i.e. the XOR of the indices of the bits equal to 1. Compute `y = w ⊕ f(a)` and flip the y-th bit of a (if y = 0, leave the cell intact). Small example: (n,k) = (3,2), a = 101, w = 11. f(a) = 1 ⊕ 3 = 01 ⊕ 11 = 10 (= 2); y = 11 ⊕ 10 = 01 (= 1); flip bit 1 → a′ = 001; f(a′) = 3 = w ✓. The receiver only computes f(a′).

Efficiency formulas

- [2]: p message bits in 2^p − 1 coefficients with at most one change and on average 1 − 2^−p changes. Relative payload α_p = p/(2^p − 1); efficiency e_p = p/(1 − 2^−p) bits per non-zero AC coefficient.
- [4]: change density D = 1/(n+1), embedding rate R = k/n, efficiency E = R/D = ((n+1)/n)·k, with n = 2^k − 1 in F5.

(n,k)	D (%)	R (%)	E (bits per change)
(1,1)	50	100	2
(3,2)	25	66.67	2.67
(7,3)	12.5	42.86	3.43
(15,4)	6.25	26.67	4.27
(31,5)	3.13	16.13	5.16
(63,6)	1.56	9.52	6.09
(255,8)	0.39	3.14	8.03

(From Table 1 of [4].) Reading: as k grows, efficiency grows but the rate falls; "the cost of higher efficiency is the lower rate" [4].

Hamming (7,4) connection (outside the papers, general coding-theory knowledge): the H above is the parity-check matrix of the Hamming(7,4) code (7-bit codeword: 4 data bits + 3 parity bits). [2] calls it "binary Hamming codes"; the label "(7,4)" does not appear in the

papers. If you use it in your presentation, support it with a coding-theory textbook reference. Intuition: a Hamming code corrects a single error; here, a single bit change can produce any syndrome, and the syndrome is the message.

Wet paper codes and STC

- **Wet paper codes** [2]: the sender may use side information the receiver does not have (e.g. which coefficients became zero). The equation is `Dy = m`, with D a shared random binary p×n matrix; positions that cannot be modified ("wet") keep x_i = y_i. This gives a continuous family of codes, more flexible than the discrete set of Hamming codes. It removes F5's shrinkage problem (nsF5).
- **Syndrome-Trellis Codes (STC)** [3]: the standard of modern adaptive steganography, "embedding cost function + steganographic embedding code". `msg = y·H^t`, where H is built from small sub-matrices placed along the diagonal (height h, width w); a Viterbi-style search finds a low-cost y. h is typically 8 to 12 [3].

7. Fan et al.: extended matrix encoding [4]

Problem. In standard matrix encoding n = 2^k − 1 is mandatory, so the cell length grows exponentially with k and the rate drops. The extension tries to increase k with n fixed.

Idea (layer extension). The coding triple (d_max, n, k) becomes a quad (d_max, n, k, L). The hash output index is multiplied by 2^l to open extra layers. Even if the cell has only n bits, only specific "extended codes" can be embedded (e.g. for a 3-bit cell, with 1-layer extension: 000, 010, 100, 110). The sender starts from the longest secret code w and finds the largest layer l with mod(w, 2^l) = 0; if none, it falls back to standard matrix encoding.

Formulas. Hash: `f(a) = ⊕ a_i·(i·2^l_crt)`; index: `y = (w ⊕ f(a)) / 2^l_crt`. The receiver must know the layer, so a **symbol** s of m = ⌈log2(L+1)⌉ bits is appended to the cell (stego cell has n + m bits).

Example (1,7,3,2). a = 1101010, w = 11001. mod(11001, 2^2) ≠ 0, so try the shorter w = 1100; mod(1100, 2^1) = 0, so the layer is l = 1. Symbol length m = ⌈log2 3⌉ = 2, s = 01. f(a) = 0010, y = (1100 ⊕ 0010)/2 = 7; flip bit 7 → a′ = 1101011; stego cell c = 110101101. The receiver reads s = 01 → l = 1, then computes f(a′) to get w = 1100.

Results and limits.

- The embedding rate can theoretically exceed 100% [4].
- Efficiency is higher than F5 when k/n is large, but lower when k/n is small (e.g. below 3/7 for L = 7) because of symbol bits [4].
- Setting the symbol costs on average m/2 changes that carry no message [4].

- The experiments use a binary logo with long runs of black pixels; for random messages, extension is possible only with probability 0.5 [4]. The gain therefore depends on the message structure. This is an honest limitation to state in your talk.
- Test: Lena 256×256, 64×64 logo; PSNR 70.33 [4].

8. Detectability of JPEG steganography: findings of [2]

Setup. 6006 raw images, grayscale JPEG, QF 70 (QF 85/70 double compression for PQ). Steganalysis: 274-dimensional features (193 extended DCT + 81 Markov) with a Gaussian-kernel SVM. Metric: error probability P (higher means more secure; 50 means random guessing). Payload unit: bpac.

Method	P @ 0.05 bpac	P @ 0.20 bpac
Steghide	2.58%	0.26%
MBS	4.61%	0.36%
JP Hide&Seek	4.01%	2.11%
PQ	6.73%	3.19%
–F5	7.37%	0.12%
F5	12.94%	0.80%
nsF5	28.90%	3.21%
MM2	38.26%	0.82%
PQe	37.80%	16.15%
PQt	40.06%	15.27%
MM3	40.86%	1.24%

Conclusions [2]:

- Against the best blind steganalysis available then, the average safe capacity is about **0.05 bpac** (grayscale, QF 70).
- Steghide, MBS and JPHS are the most detectable. **Statistics-preserving** methods (Steghide, MBS) lose to heuristically designed methods that use matrix embedding (F5, nsF5).
- **Matrix embedding** markedly improves security by diluting the number of changes; its importance grows as payload decreases.

- The best method without side information is nsF5; methods with side information (uncompressed cover) are PQt and MM3.
- Content-adaptive selection (texture- and energy-based PQt/PQe) improves security, but the selection rule may leak information to the attacker.
- The **JPEG compressor** matters: preparing covers with a different compressor can artificially raise nsF5's detectability by up to 15% for short messages.

Note: these results are from 2007; newer adaptive methods (e.g. J-UNIWARD) are mentioned in [3].

9. JPEG-robust steganography [3]

Motivation. Social media (Facebook, WeChat, Instagram) recompress images as JPEG because of bandwidth and other limits. This lossy operation destroys messages hidden by traditional methods. Robust steganography should survive the lossy channel while still resisting steganalysis.

Background. Adaptive steganography = **embedding cost function + STC** (e.g. J-UNIWARD, UED). It works well in lossless channels but fails in lossy ones. DCRAS and FRAS are robust but need the QF of the attacking compression (side information).

Proposed method (no QF knowledge):

1. **Error-diffusion analysis:** the bigger the height h and width w of the STC sub-matrix, the higher the error probability of the extracted message; one stego-bit error affects h message bits.
2. **Robust virtual cover:** for each 8×8 block, the DCT coefficient D_k(i) is compared with the mean of the same-position coefficients in three neighboring blocks; larger gives 1, otherwise 0. The message is embedded into this virtual bit sequence. Because it is relational, it is more stable under compression.
3. **Cost function:** J-UNIWARD costs adapted to the virtual cover.
4. **Reducing error diffusion:** lower the embedding rate (smaller w), choose low-frequency positions, and apply **Reed–Solomon (6,15)** error correction.

Results. BOSSbase 1.01 (10,000 images), cover QF 75, attacks at QF 65 to 85: extraction error ≈ 0.3–3% (the compared robust watermarking method: 3–10%). Against steganalysis (ccPEV, DCTR) it is at most 7% worse than DCRAS; the authors attribute this to STC working on only part of the cover. This section ties directly to the "image distortion vs steganography" topic.

10. Deep-learning steganography, short map [1]

[1] groups methods into three classes:

- **Traditional:** LSB, PVD, DCT/DWT, EMD, etc. Low capacity, lower security.
- **CNN-based:** encoder–decoder (U-Net, VGG-based). Cover and secret image are combined into a stego image; a decoder recovers the secret. Very high capacity (close to 1).
- **GAN-based:** generator + discriminator (+ steganalyzer); SGAN, SSGAN, HiDDeN, CycleGAN variants, coverless methods. Best security and PSNR results (e.g. 64.7 dB for a CycleGAN-based method).

Metrics: PSNR/MSE/SSIM, steganalysis accuracy, BPP. Datasets: BOSSBase (10,000 grayscale 512×512), ImageNet, CelebA, COCO, etc. Open problems: no common benchmark, GAN convergence and mode collapse, real-time use, robustness to man-in-the-middle and tampering [1].

11. Summary table for your talk: method → idea → weakness

Method	Domain	Idea	Weakness
LSB	Spatial	Replace pixel LSBs with message bits	Visual/statistical attacks, 50% change rate, dies in lossy channels
Dynamic-key LSB	Spatial	PRNG-selected positions	Still carries LSB's statistical footprint
PVD + LSB	Spatial	Pixel difference decides bit count	Texture-dependent capacity
JSteg	DCT	LSB of DCT coefficients, skip 0/1	POV, χ^2 and histogram attacks
Outguess	DCT	PRNG scatter, statistical restoration	Blockiness attack, restoration footprint
F3	DCT		coef
F4	DCT	Sign-aware mapping +	coef
F5	DCT		coef
nsF5	DCT	F5 + wet paper codes	No side information; less secure than PQ/MMx
Extended ME [4]	DCT	Layer extension, rate over 100%	Efficiency drops for small k/n; message-dependent

Method	Domain	Idea	Weakness
Bao et al. [3]	DCT	Robust virtual cover + STC + RS	Slightly higher detectability than DCRAS

(The F4 weakness cell is a synthesis: [4] only states that F4′s histogram looks clean; it does not list a weakness.)

12. Likely questions in the presentation

- **Why is decreasing |coef| better than flipping the LSB?** LSB flipping equalizes the pair counts (2k, 2k+1) and leaves a footprint; decreasing does not make this symmetric equalization [2], [4].
- **Why does matrix embedding improve security?** Fewer changes, i.e. lower change density and higher embedding efficiency [2], [4].
- **What is shrinkage and why is it a problem?** The receiver cannot see a coefficient that became zero, so the bit is embedded again; efficiency and histogram suffer [2].
- **How much can you hide safely?** About 0.05 bpac [2].
- **What happens on social media?** Recompression destroys the message; robust methods use a virtual cover, STC and error correction [3].
- **What are newer approaches?** CNN/GAN-based methods [1].

13. Citation notes

- Only statements based on [1]–[4] are cited. The label "Hamming(7,4)" and the single-error-correcting property of the Hamming code are general coding-theory knowledge outside the papers; back them with a coding-theory source if you use them.
- If you want to cite primary sources mentioned inside the papers (Westfeld′s F5, Crandall, Sallee′s MBS, Filler′s STC, Fridrich′s wet paper codes, etc.), take the full entries from the reference lists of the corresponding paper. Here they are shown through [1]–[4] as secondary sources.